Kubernetes Readiness Probe

A **Readiness Probe** is a Kubernetes mechanism used to determine whether a **Pod is ready to receive network traffic**.

In simple terms:

Readiness Probe answers: "Can this Pod safely handle requests right now?"

If the readiness probe fails, Kubernetes **does not normally restart the container**. Instead, Kubernetes removes that Pod from the endpoints used for Service traffic until the Pod becomes ready again.

1. Why do we need a Readiness Probe?

A container can be running but not ready to serve requests.

For example, imagine a Java application:

Pod

└── Java Application

  ├── JVM → Running

  ├── Process → Running

  ├── Application → Starting

  └── Database connection → Not established

Kubernetes might see:

Container → Running

But sending user traffic to it immediately could result in:

User

↓

Service

↓

Pod

↓

Application

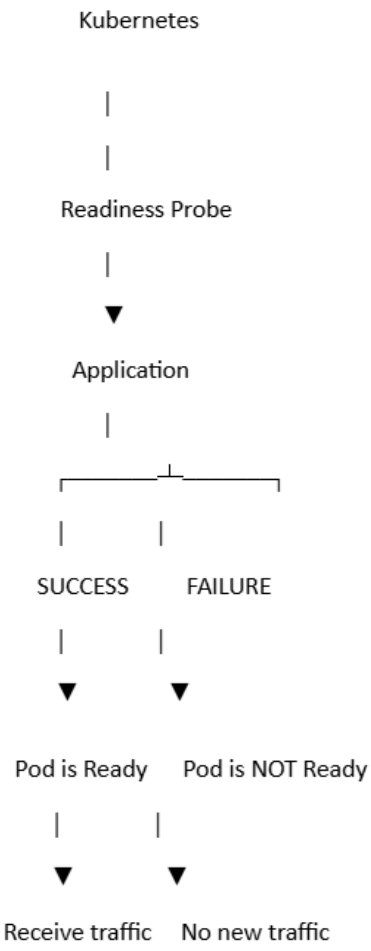
↓

ERROR

A readiness probe prevents Kubernetes from sending traffic to the Pod until the application is actually ready.

2. Basic Concept

The flow is:



The key point is:

Readiness controls traffic, not container lifecycle.

3. Readiness vs Liveness

This is one of the most important Kubernetes interview concepts.

Question	Liveness	Readiness
What does it check?	Is application alive?	Can application serve traffic?
Main purpose	Recovery	Traffic management
Probe fails	Container may restart	Pod removed from Service endpoints
Container restarted?	Yes, after threshold	No
Can Pod remain running?	Generally yes, but container is restarted	Yes
Example problem	Deadlock/hang	Application starting / temporarily unavailable

Think of it this way:

Liveness:

"Should Kubernetes restart me?"

Readiness:

"Should Kubernetes send users to me?"

4. Pod Ready vs Container Running

This distinction is extremely important.

A Pod can be:

Container:

Running ✅

Pod:

Not Ready ❌

There is no contradiction.

For example:

Pod

|

├— Container → Running

|

└── Readiness → Failed

The application process is alive, but it isn't currently suitable for receiving traffic.

Practical 3 — Readiness Probe

Now create a Service and see the difference between **readiness and liveness**.

```
apiVersion: v1
kind: Pod
metadata:
  name: readiness-demo
  labels:
    app: readiness-demo
spec:
  containers:
   - name: nginx
     image: nginx:1.27


     ports:
      - containerPort: 80


     readinessProbe:
       httpGet:
         path: /
         port: 80
       initialDelaySeconds: 5
       periodSeconds: 5
```

Save:

readiness.yaml

Apply:

kubectl apply -f readiness.yaml

Check:

kubectl get pod readiness-demo

Practical 4 — Create a Service

apiVersion: v1

kind: Service

metadata:

name: readiness-service

spec:

selector:

app: readiness-demo

ports:

- port: 80

targetPort: 80

Save:

service.yaml

Apply:

kubectl apply -f service.yaml

Check:

kubectl get svc

Now check endpoints:

kubectl get endpoints readiness-service

Or, on newer Kubernetes versions:

kubectl get endpointslices

Because the Pod is ready, its IP should appear as a Service endpoint.

Practical 5 — Make Readiness Fail

Change:

path: /

to:

path: /health

Apply:

kubectl apply -f readiness.yaml

Check:

kubectl get pod readiness-demo

You'll see something similar to:

READY

0/1

But notice:

kubectl get pod readiness-demo

The Pod can still be:

STATUS

Running

This is the **important difference**.

Readiness failure

Application running

↓

Readiness probe fails

↓

Pod becomes NotReady

↓

Removed from Service endpoints

↓

NO traffic sent

The container is **not restarted**.

5. What happens when Readiness Probe fails?

Suppose you have:

Service

|

├── Pod A → Ready

├── Pod B → Ready

└── Pod C → Not Ready

The Service will normally route traffic only to:

Pod A

Pod B

Pod C remains running:

Pod C

|

├── Container → Running

└── Readiness → Failed

When the readiness probe becomes successful again:

Pod C

|

└── Readiness → Success

↓

Ready

↓

Added back to Service endpoints

Traffic can then be sent to Pod C again.

6. Readiness is Dynamic

Readiness isn't necessarily a one-time check.

A Pod can transition between:

Ready

↓

Not Ready

↓

Ready

↓

Not Ready

For example:

Application healthy

↓

Ready

↓

Temporary dependency problem

↓

Not Ready

↓

Problem resolved

↓

Ready

This is useful for applications that may temporarily become unable to handle traffic without needing to be restarted.

7. Readiness During Application Startup

Consider a Spring Boot application:

Pod starts

↓

JVM starts

↓

Spring Boot starts

↓

Beans initialized

↓

Configuration loaded

↓

Connections initialized

↓

Application ready

During startup:

Container → Running

Pod → Not Ready

Once the readiness condition succeeds:

Pod → Ready

Only then should the application receive normal Service traffic.

This prevents the classic problem:

Pod starts

↓

Service immediately sends traffic

↓

Application still starting

↓

Requests fail

8. Readiness During Rolling Updates

Readiness probes are especially important during a **Deployment rolling update**.

Imagine:

Deployment

├── Pod A → Version 1 → Ready

├── Pod B → Version 1 → Ready

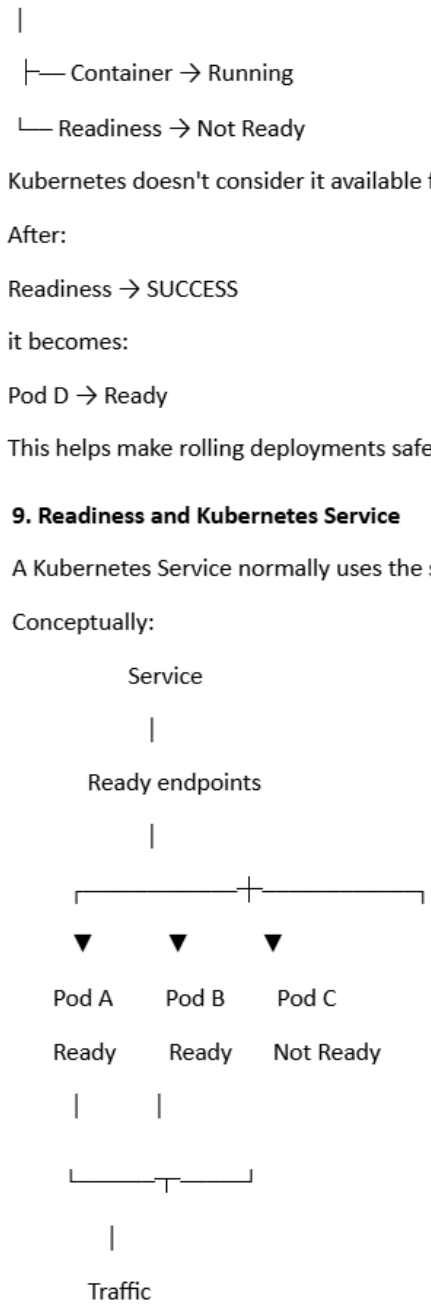
└── Pod C → Version 1 → Ready

A new version is deployed:

New Pod D → Version 2

Pod D starts:

Pod D



Kubernetes doesn't consider it available for normal Service traffic until it becomes ready.

After:

Readiness → SUCCESS

it becomes:

Pod D → Ready

This helps make rolling deployments safer.

9. Readiness and Kubernetes Service

A Kubernetes Service normally uses the set of **ready endpoints** for routing traffic.

Conceptually:

Pod C is still running, but isn't included as a normal traffic destination while it is not ready.

10. Readiness Probe Types

The same basic probe mechanisms used for liveness can be used for readiness.

HTTP

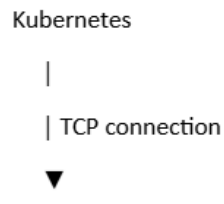
Kubernetes sends an HTTP request to an endpoint.

Conceptually:



TCP

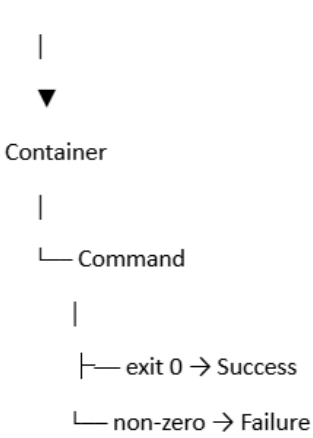
Kubernetes attempts to establish a TCP connection to a port.



If the connection succeeds, the probe can succeed.

Exec

Kubernetes executes a command inside the container.



gRPC

For gRPC applications, Kubernetes can perform a gRPC health check.

Kubernetes

|

| gRPC health check

▼

gRPC application

11. Important Readiness Probe Parameters

Readiness probes have timing and threshold settings.

initialDelaySeconds

How long Kubernetes waits before starting the probe.

Conceptually:

Container starts

↓

Wait

↓

Initial delay

↓

Readiness checks begin

periodSeconds

How frequently Kubernetes performs the readiness check.

For example:

periodSeconds = 10

means checks occur approximately every 10 seconds.

timeoutSeconds

How long Kubernetes waits for a probe response.

If the application doesn't respond within the configured timeout, that probe attempt can be considered a failure.

---

failureThreshold

Number of consecutive failures required before the readiness state changes to failure.

For example:

Failure #1 → Not necessarily enough

Failure #2 → Not necessarily enough

Failure #3 → Mark Not Ready

---

successThreshold

Number of consecutive successful probes required before the Pod is considered ready again.

For readiness probes, this can be greater than 1.

For example:

successThreshold = 2

Conceptually:

Success #1 → Not Ready yet

Success #2 → Ready

This can help avoid rapidly switching between Ready and Not Ready because of transient conditions.

12. Readiness Probe During Temporary Failure

Suppose an application temporarily cannot process requests.

The readiness probe starts failing:

Application

↓

Readiness failure

↓

Pod becomes Not Ready

↓

Service stops sending new traffic

Importantly:

Container → Still Running

Once the application recovers:

Readiness success

↓

Pod becomes Ready

↓

Service can send traffic again

13. Readiness Probe Should Represent Traffic Readiness

The probe should answer:

"If I send a request to this Pod right now, can the application reasonably handle it?"

This is different from asking:

"Is the process alive?"

For example:

Process alive?

↓

YES

doesn't automatically mean:

Ready to receive traffic?

↓

YES

14. Readiness and External Dependencies

This is where architecture becomes important.

Suppose:

Application

|

├── Database

├── Redis

├── Kafka

└── External API

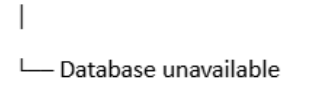
Should the readiness probe check all of them?

Not necessarily.

It depends on whether the application can meaningfully serve requests without each dependency.

For example:

Application



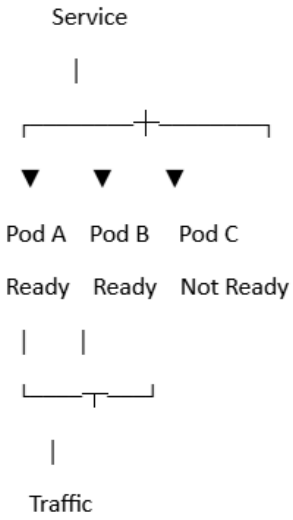
If the application cannot serve any meaningful requests without the database, marking it **Not Ready** may be appropriate.

But if the application can still serve some requests or queue work while the database is unavailable, blindly making the whole Pod Not Ready may be undesirable.

So readiness should reflect the application's **actual traffic-serving requirements**, not simply every dependency being healthy.

15. Readiness Probe and Load Balancing

Consider three Pods:



The Service has two suitable destinations:

Pod A

Pod B

Pod C can remain running without receiving new Service traffic.

This makes readiness particularly important for:

- Rolling deployments
- Application startup
- Temporary application unavailability
- Graceful maintenance
- Dependency-related availability conditions
- Stateful or initialization-heavy applications

16. Readiness and Graceful Shutdown

Readiness is also important when a Pod is being terminated.

Conceptually:

Pod receives termination

↓

Pod becomes unavailable for new traffic

↓

Existing work gets time to finish

↓

Container terminates

This helps avoid sending new requests to a Pod that is in the process of shutting down.

In real production systems, readiness, termination behavior, preStop, terminationGracePeriodSeconds, and application-level graceful shutdown can work together.

17. Readiness Probe Does Not Restart Containers

This is probably the **#1 interview point**.

If:

Readiness Probe → FAILURE

the normal consequence is:

Pod → Not Ready

not:

Container → Restart

Compare:

Liveness failure

↓

Container restart

with:

Readiness failure

↓

Remove from traffic

Liveness vs Readiness

Liveness failure

Pod

↓

Liveness fails

↓

Container restarted

Readiness failure

Pod

↓

Readiness fails

↓

Pod remains running

↓

Removed from Service endpoints

18. Readiness vs Startup Probe

Startup and readiness solve different problems.

Startup

"Has my application finished starting?"

Readiness

"Can my application receive traffic now?"

For example:

Container starts

↓

Startup Probe

↓

Application initialization

↓

Startup successful

↓

Readiness Probe

↓

Application ready

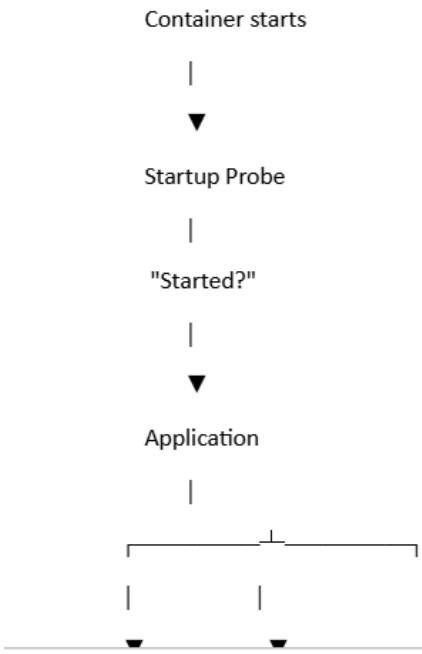
↓

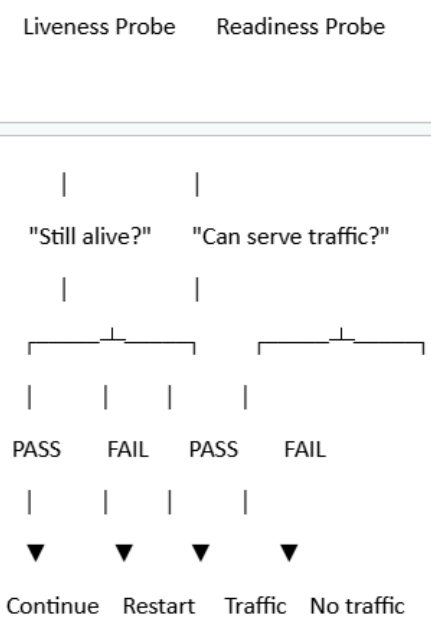
Service traffic

> A slow application might take a long time to start but still need an additional readiness condition after startup.

19. Readiness vs Liveness vs Startup — Complete Picture

A useful mental model is:





20. Common Misconceptions

❌ **"Readiness failure restarts the Pod."**

Not normally.

✅ **Correct:**

Readiness failure

↓

Pod becomes Not Ready

↓

Traffic is removed

❌ **"Running means Ready."**

Not necessarily.

Running ≠ Ready

A container can be running while the Pod is not ready.

❌ **"Readiness is only checked once."**

No.

Kubernetes periodically checks the readiness condition.

❌ **"Readiness is only useful during startup."**

No.

A Pod can become Not Ready at any point during its lifetime.

❌ **"Readiness and liveness should use exactly the same endpoint."**

Not necessarily.

They answer different questions and often should have different semantics.

---

21. Interview Definition

A readiness probe is a Kubernetes health-check mechanism used to determine whether a Pod is currently capable of receiving traffic. Kubernetes periodically executes the configured readiness check using mechanisms such as HTTP, TCP, gRPC, or an exec command. When the probe fails, Kubernetes marks the Pod as Not Ready and removes it from the Service's ready endpoints, so new traffic is not routed to that Pod. The container is not restarted simply because a readiness probe fails. When the probe succeeds again according to the configured thresholds, the Pod becomes Ready and can receive traffic again.

The easiest way to remember all three

Probe	Think of this question
🟢 Startup	"Have I started?"
🔵 Readiness	"Can I receive traffic?"
🔴 Liveness	"Should I be restarted?"

Startup protects the application during startup.
Readiness protects users from unhealthy/unready Pods.
Liveness helps Kubernetes recover stuck containers.